

FULL SEO AUDIT REPORT

dealdost.shop

Prepared by Claude SEO

20/₁₀₀

SEO Health Score

August 03, 2026 Full Audit

Table of Contents

1. Executive Summary

Key Metrics, Critical Issues & Quick Wins

2. Audit Categories

What Works

Findings by Severity

3. Action Plan

Phased Roadmap

4. Data Sources & Methodology

1. Executive Summary

This report presents a comprehensive SEO audit of **dealdost.shop** for a **E-commerce / Affiliate deals aggregator (Amazon & Flipkart deals sourced from Telegram)** site, generated on August 03, 2026. Findings combine technical, content, schema, performance, visual, and search-readiness evidence as available.

20/100

SEO HEALTH SCORE

Critical Issues Found:

1. **Info:** Production is serving the raw Vite DEV server (not a build) — `/@vite/client`, `/@react-refresh` HMR injections and unbundled `/src/main.tsx` all return HTTP 200 live
2. **Info:** `robots.txt` and `sitemap.xml` both return HTTP 200 but serve the SPA's `index.html` shell — neither file is reachable, and no static asset in `apps/web/public/` is served at all
3. **Info:** No per-deal/product permalink pages exist anywhere (no `/deal/:id` route) — every deal is a card inside a listing page with no unique, indexable, shareable URL
4. **Info:** Zero structured data (JSON-LD) reaches the live site, despite `Organization/WebSite` schema already being coded in `Home.tsx` — blocked purely by the dev-server deployment issue
5. **Info:** Zero FTC/ASCI affiliate disclosure anywhere; no About, Privacy, Terms, or Contact pages exist in the codebase at all

Quick Wins:

1. Deploy ``pnpm --filter @deals/web build`` output behind nginx instead of running the Vite dev server — single highest-leverage fix, unblocks meta tags, canonical, favicon/manifest, and the already-coded `Organization/WebSite` JSON-LD for free
2. Fix nginx to serve static files (`robots.txt`, `sitemap.xml`, icons) before the SPA catch-all — the repo already has reasonably well-formed `robots.txt/sitemap.xml`, they just never reach a crawler
3. 301-redirect `www.dealdost.shop` to the apex domain (TLS cert already covers both hosts)
4. Add standard security headers (HSTS, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) via nginx `add_header`
5. Remove `loading="lazy"` from the first above-the-fold product card image (current LCP element)

2. Audit Categories

2.1 Technical SEO

Score: **22/100**

What Works

- HTTPS enforced correctly with a valid 301 redirect from http:// to https://
- TLS certificate covers both apex and www hosts
- Mobile viewport meta tag present and correct on all routes
- Content is eventually renderable by a JS-executing crawler (Playwright did extract real product text after a long render) — not a hard JS-blocking wall, purely a performance/timeout risk

Findings

Production is running the Vite dev server, not a production build **Critical**

https://dealdost.shop/ returns raw dev-server HTML: `/@react-refresh` HMR bootstrap, `@vite/client">`, and unbundled `/src/main.tsx` returns HTTP 200 directly. No hashed `/assets/\*.js` production bundle exists anywhere. Cache-Control: no-cache is set site-wide.

Recommendation: Run `pnpm --filter @deals/web build` to produce apps/web/dist/, and serve that static output via nginx instead of the vite/pnpm dev process.

No robots.txt or sitemap.xml reach crawlers **Critical**

Both /robots.txt and /sitemap.xml return HTTP 200 but the body is the SPA's index.html shell. sitemap_discovery.py found declared:[] found:[] — nothing valid exists. Files already exist in apps/web/public/ but the deployed dev server doesn't serve static assets at all.

Recommendation: Fix nginx/static routing so real files in apps/web/public/ (or dist/) are served before the SPA catch-all; redeploy the production build.

No canonical tag, meta description, or per-route title anywhere in raw HTML **Critical**

Every one of 9 routes tested returns byte-identical <head> content (same Content-Length: 709, same ETag). Seo.tsx (react-helmet-async) is coded to set these client-side, but given the ~17s render time, crawlers may never execute the JS that sets them, and Google deprioritizes JS-injected rel=canonical.

Recommendation: Resolves automatically once the production build is deployed (index.html already has static description/canonical in the repo). Verify each route gets a distinct Seo.tsx title/canonical post-fix.

No indexable per-deal/product URLs **Critical**

Routes are limited to /, /category/:name, /digest, /search, /trending, /price-drops, /assistant, /login, /signup. No /deal/:id or /product/:id route exists — individual deals only render as cards inside listing pages.

Recommendation: Add a /deal/:id detail route with server-renderable-equivalent meta tags per deal; include in the sitemap. Backend data (GET /v1/deals/:id, GET /v1/products/:id) already supports this.

www.dealdost.shop serves duplicate content with no redirect and no canonical **High**

https://www.dealdost.shop/ returns 200 OK with byte-identical body/ETag to the apex domain. TLS cert covers both hosts so there's no blocker to fixing this at the nginx/DNS level.

Recommendation: Add an nginx server block that 301-redirects www.dealdost.shop to https://dealdost.shop for all paths.

Missing all standard security headers **High**

Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy are all absent from every response.

Recommendation: Add via nginx add_header: HSTS, X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin, and a CSP scoped to real script/style/API origins once dev-mode injections are removed.

No structured data (Schema.org) anywhere on the live site **High**

Playwright-rendered structured-data extraction returned block_count: 0 on every route checked, despite visible product cards in rendered text.

Recommendation: See Schema category — Organization/WebSite JSON-LD is already coded and should appear automatically once the production build ships; add BreadcrumbList and ItemList as new work.

Referenced favicon/manifest assets soft-404 to the SPA shell **Medium**

/icon-192.png and /manifest.json both return HTTP 200 with Content-Type: text/html (the SPA shell) instead of image/png or application/json. The deployed index.html references stale asset paths that predate the current repo's favicon.svg/apple-touch-icon.png.

Recommendation: Resolves automatically once the current build is deployed. Separately, configure nginx to return a real 404 for unmatched static-looking paths (*.png, *.json, *.ico).

Soft-404s: unknown paths return 200 with the homepage shell **Medium**

GET /nonexistent-page-xyz returns 200 OK, byte-identical to /, with no noindex signal in the raw HTTP response.

Recommendation: Ensure the client router's not-found state sets noindex via Seo.tsx, and verify it fires given current rendering performance.

2.2 Content Quality

Score: **15/100**

What Works

- Above-the-fold copy (once rendered) is clear and value-prop focused
- Real, differentiated product data (price, discount %, price-drop %, ratings, coupon text, bank offers) already exists in the backend and is richer than most competitors expose

Findings

No individual deal/product pages exist **Critical**

Deals only appear as DealCard components inside listing pages. DealCard fires the buy action via onClick, not an <a href>, so there isn't even an anchor to expose. This is the root cause of nearly every other content finding.

Recommendation: Introduce a permalink per deal (/deal/:slug-or-id) with title, price, image, source badge, and stable canonical URL.

SEO metadata is coded but never reaches the live page **Critical**

Seo.tsx (react-helmet-async) is wired into Home.tsx and Category.tsx to inject per-route title/description/canonical/OG/JSON-LD, but full-render checks on three routes all returned the identical static head. Directly caused by the dev-server-in-production issue.

Recommendation: Fix production deployment (see Technical); re-verify Helmet output renders per-route afterward.

No FTC/ASCI affiliate disclosure anywhere on the site **Critical**

Zero matches for 'disclosure', 'affiliate', 'About', 'Contact', 'Privacy', 'Terms', 'FTC', or 'commission' anywhere in rendered HTML/body text. No footer exists. No /about, /disclosure, /privacy, /terms, or /contact route exists in App.tsx at all.

Recommendation: Add a footer present on every page with an affiliate disclosure statement, About blurb, Privacy Policy, Terms, and a real contact method. Compliance issue, not just SEO.

Category pages carry no unique content beyond a templated one-line heading **High**

Rendered text for /category/electronics after full JS render was just nav chrome plus a single un-capitalized heading ("electronics Deals") with no supporting copy.

Recommendation: Add a short (100-200 word) category intro — what appears here, update cadence, curation criteria — written once per category, not templated.

Deal 'content' is verbatim marketplace copy with no original commentary High

product_title is copied directly from the first line of the raw Telegram message (itself the marketplace's own listing title) with no rewriting or added commentary. No Experience or Expertise signal per Google's Sept 2025 QRG.

Recommendation: Add one original sentence per deal at ingestion/curation time using existing previous_price/price_drop_percent data rather than relying solely on the scraped title.

No Product/Offer/ItemList schema exists even in source code High

Home.tsx's JSON-LD only covers Organization and WebSite. No per-deal schema exists, and there's no permalink URL to anchor it to yet.

Recommendation: Add Product+Offer JSON-LD once per-deal pages exist; add ItemList to listing pages in the meantime.

Inconsistent client-side hydration blocks crawler visibility on some routes Medium

/trending fully rendered its product grid within the Playwright render budget; /category/electronics did not, showing only nav + heading after ~16s.

Recommendation: Investigate route-to-route fetch/render timing differences; prioritize SSR or static pre-rendering for listing pages.

Duplicate content risk from unmodified marketplace titles Medium

Because product_title is copied verbatim from source listings, the same title strings very likely appear across many other deal-aggregator sites/Telegram channels republishing the same feeds.

Recommendation: Covered by the original-commentary fix plus stable per-deal URLs — treat as one connected fix.

2.3 On-Page SEO

Score: **15/100**

What Works

- index.html in the current repo (not yet deployed) already has a correct meta description and canonical tag
- Category page title/description template (Seo.tsx) is reasonable, just not yet differentiated with real content

Findings

Identical <title> and no meta description on every route in raw HTML **Critical**

All 9 routes tested return the exact same 709-byte head with no per-route title variation, poor for CTR and self-competing pages in SERPs.

Recommendation: Ship the production build so Seo.tsx's per-route titles/descriptions render; add a static fallback description/canonical in index.html itself as already exists in the repo source.

No heading hierarchy (0 H1s found on rendered pages in SXO testing) **High**

SXO audit found 0 <h1>, 0 <h2> on rendered / and /price-drops after full JS execution.

Recommendation: Ensure each page template renders a real <h1> (page/category title) as part of the static or SSR'd markup, not solely client-injected.

No internal linking to individual deals **High**

DealCard uses onClick handlers instead of <a href> links, so there is no crawlable internal link graph pointing at individual products.

Recommendation: Once per-deal permalinks exist, link DealCard titles to them with real <Link>/<a> elements in addition to the buy button.

2.4 Schema & Structured Data

Score: **10/100**

What Works

- Organization + WebSite (with SearchAction) JSON-LD is already correctly authored in Home.tsx and passes schema validation checks
- Seo.tsx has a working jsonLd prop mechanism via react-helmet-async, and HelmetProvider is correctly wired in main.tsx

Findings

Zero JSON-LD/Microdata/RDFa reaches the live site on any route **Critical**

structured_data.block_count is 0 on /, /trending, /category/electronics, and /price-drops, both raw and rendered. This is a deployment issue, not a schema-authoring issue — the code exists but the dev-server-in-production issue prevents it from shipping.

Recommendation: Fix the production deployment (see Technical). Organization/WebSite schema should appear for free once fixed; re-crawl to confirm.

No BreadcrumbList on category pages **High**

Category.tsx renders /category/:name with no breadcrumb schema.

Recommendation: Add BreadcrumbList (Home > Category) via the existing jsonLd prop on <Seo> in Category.tsx — trivial effort.

No ItemList schema on listing pages **Medium**

Home, Trending, PriceDrops, and Category pages show deal grids with no ItemList markup describing them. Without per-deal URLs, this won't qualify for carousel rich results, but is still valuable entity/context data.

Recommendation: Add ItemList with nested Product/Offer (no item.url yet) populated from the same deal array already rendered by InfiniteDealGrid/DealCard.

No Product/Offer schema with rich-result eligibility (dependency, not blocking) **Info**

Full Product/Offer schema requires a per-deal permalink URL to anchor to, which doesn't exist yet.

Recommendation: Once /deal/:id pages ship, add Product + Offer JSON-LD (price, currency, availability, image, seller) using fields already present on the Deal/Product type.

2.5 Performance (Core Web Vitals)

Score: **54/100**

What Works

- CLS = 0 on both routes tested — no layout-shift issues
- TBT (INP lab proxy) is 120-130ms, well under the 200ms 'good' threshold — main-thread JS execution itself is not heavy

Findings

LCP is 16.1s (home) / 17.2s (trending) — ~6.5x over the 'poor' threshold **Critical**

Lighthouse mobile lab run: LCP 16,128ms / 17,168ms against a 2,500ms 'good' threshold. 90% of the LCP time on /trending is 'Load Delay' — the browser resolving unbundled dev-mode JS module chains before it even starts fetching the LCP image. FCP (9.0-9.3s), Speed Index (12.8-12.9s), and TTI (18.7-19.4s) are all similarly poor. Lighthouse Performance score: 54/100 on both routes.

Recommendation: Deploy the production `vite build` output instead of the dev server. Expected to cut LCP from ~16-17s to low single digits on its own.

Static JS assets served without compression **High**

The largest JS chunk (928KB, unminified) returns HTTP 200 with no Content-Encoding header despite the client advertising gzip/br support. Lighthouse estimates 1.7MB of savings from text compression alone sitewide.

Recommendation: Enable gzip/brotli in nginx for application/javascript, text/javascript, and text/css once the production build is deployed.

LCP element uses loading="lazy" despite being above-the-fold **High**

The first product card image (Lighthouse's identified LCP element on /trending) has loading="lazy", delaying its fetch until layout completes.

Recommendation: Use loading="eager" plus fetchpriority="high" for the first N above-the-fold cards; keep lazy-loading for the rest of the grid.

TTFB is borderline (230ms / 744ms observed) **Medium**

Root document TTFB is right at or above the 200ms 'good' threshold on both routes; likely reflects backend/cold-start latency rather than static-asset serving.

Recommendation: Re-check after the production build ships; consider short-TTL response caching on feed endpoints given the 10-minute poller cadence.

2.6 Images

Score: **35/100**

What Works

- Product images render correctly once the page loads and are appropriately sized in the layout (no CLS from images)
- Alt text exists via the product title, even if not always clean/descriptive

Findings

Product images are hotlinked directly from Amazon/Flipkart with no local optimization or CDN control **Low**

deal.image_url is rendered directly with no proxy/CDN rewrite and no explicit width/height attributes.

Recommendation: Set explicit width/height on `<img>` to guard against CLS; consider an image proxy/cache as a defensive measure against source images breaking or rate-limiting.

LCP candidate image is lazy-loaded (cross-referenced with Performance) High

See Performance category — the first above-the-fold product image uses `loading="lazy"`, directly harming LCP.

Recommendation: Switch to `eager/fetchpriority=high` for the first visible card image.

Alt text is raw scraped marketplace title, not always clean Low

Titles sometimes include marketplace boilerplate ("Buy X Online at Best Price...") rather than clean descriptive text.

Recommendation: Reuse existing title-cleaning logic (used for search indexing) for displayed/alt text as well.

2.7 AI Search Readiness (GEO)

Score: **10/100**

What Works

- Once fully rendered, Playwright does successfully extract real product text, meaning Google's JS-executing systems are not entirely blind to content — the ceiling is render-time, not a hard block

Findings

AI crawlers that don't execute JavaScript see nothing Critical

GPTBot, ClaudeBot, and PerplexityBot user agents were tested directly against the raw response — all received an identical 709-byte empty shell (`<div id="root"></div>` only), regardless of User-Agent.

Recommendation: Fix the production deployment first, then evaluate SSR/prerendering (e.g., `prerender-on-bot-UA` middleware) so first-response HTML contains real deal content.

No real robots.txt or llms.txt — well-known AI-crawler paths fall through to the SPA shell Critical

`/robots.txt`, `/llms.txt`, `/sitemap.xml`, and `/ai.txt` all return HTTP 200 byte-identical to the homepage shell.

Recommendation: Fix static-file serving precedence; add explicit Allow rules for GPTBot/OAI-SearchBot/ClaudeBot/PerplexityBot plus a Sitemap: directive, and an `llms.txt` summarizing the site's purpose and key routes.

No permalink URL per deal blocks AI citation structurally **Critical**

Even after fixing rendering, an AI answer engine has no stable URL to cite for a specific deal — only an aggregate listing page whose contents rotate every ~10 minutes.

Recommendation: Add /deal/:id permalink pages with server-rendered title, description, OG tags, and Product/Offer JSON-LD.

No meta description, Open Graph, or Twitter Card tags anywhere **High**

The entire <head> (raw or rendered) contains only charset, viewport, theme-color, and <title>. This also produces broken link previews when DealDost URLs are shared on Telegram/Twitter/WhatsApp — ironic given Telegram is the product's own distribution channel.

Recommendation: Add static or dynamically-injected meta description + OG/Twitter tags per route.

3. Action Plan

Phase 1: Critical Fixes Week 1

- Deploy `pnpm --filter @deals/web build` output behind nginx instead of running the Vite dev server (single highest-leverage fix — unblocks meta tags, canonical, favicon/manifest, and already-coded Organization/WebSite JSON-LD)
- Fix nginx static-file routing so real files (robots.txt, sitemap.xml, icons, manifest.json) are served before the SPA catch-all
- 301-redirect www.dealdost.shop to the apex domain
- Add a footer with FTC/ASCI affiliate disclosure, About, Privacy Policy, Terms, and Contact
- Re-crawl the site after deployment to confirm meta tags, canonical, and Organization/WebSite JSON-LD now render live

Phase 2: High-Impact Improvements Weeks 2-3

- Add /deal/:id permalink route + page (title, price, image, source badge, rating, coupon/bank offers, price-history callout, Get This Deal CTA) — backend data already exists via GET /v1/deals/:id
- Link every DealCard title to its new permalink page in addition to the existing buy button
- Add Product + Offer JSON-LD to each new deal page; add BreadcrumbList to category pages; add ItemList to listing pages
- Add standard security headers (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy) via nginx
- Enable gzip/brotli compression in nginx for JS/CSS static assets
- Remove loading="lazy" from the first above-the-fold product card image; use eager + fetchpriority=high instead
- Generate a real sitemap.xml covering static routes plus per-deal URLs once they exist (recommend generating server-side in apps/api so lastmod reflects real deal update times)

Phase 3: Content & Authority Month 2

- Add a short category-specific intro (100-200 words) to each of the 8 category pages
- Add one original sentence of curation commentary per deal using existing previous_price/price_drop_percent data
- Surface the existing feed/compare price-comparison data as static on-page content with AggregateOffer schema instead of a client-only modal
- Add llms.txt and explicit AI-crawler allow rules (GPTBot, ClaudeBot, PerplexityBot) to robots.txt
- Register free Moz and Bing Webmaster Tools accounts; submit to relevant deal/coupon directories and cross-promote with source Telegram channel admins

Phase 4: Monitoring & Iteration Ongoing

- Re-run Lighthouse/PSI after the production deploy to confirm LCP $\leq 2.5s$; pull CrUX field data once traffic accumulates
- Re-run this SEO audit after Phase 1-2 fixes ship to confirm the health score improvement and catch any regressions
- Monitor Search Console (once available) for indexation status of new /deal/:id pages
- Re-run the backlink audit in 60-90 days once outreach begins

Data Sources & Methodology

Source	Description	Update Frequency
PageSpeed Insights API	Lighthouse lab audit (mobile emulation, Moto G Power, slow 4G)	Real-time
Chrome UX Report (CrUX)	28-day rolling field data from real Chrome users	Daily ~04:00 UTC
CrUX History API	25-week p75 trend data per metric	Weekly
Google Search Console	Search Analytics (clicks, impressions, CTR, position)	2-3 day lag
URL Inspection API	Per-URL index status, coverage state, crawl info	Real-time (2,000/day)

Report generated by Claude SEO — Google SEO Intelligence Skill — August 03, 2026
Methodology based on Google Web Vitals thresholds, Search Console documentation, and Lighthouse scoring algorithms.